

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Métricas

Universidad Nacional de La Plata



Facultad de Informática





1 Métricas en los sistemas paralelos

2 Tiempo de ejecución

3 Throughput

4 Speedup

Speedup absoluto y relativo. Speedup degradado. Speedup lineal (ideal o perfecto), sublineal y superlineal: Speedup lineal (ideal o perfecto), Speedup sublineal, Speedup superlineal: Algoritmo secuencial no óptimo, diferencias en la carga de trabajo, efectos debidos a la arquitectura.

5 Eficiencia

6 Balance de carga

7 Overhead

8 Escalabilidad

Ley de Amdahl: Ejemplo de la ley de Amdahl, Formalización matemática. Ley de Gustafson-Barsis: Ejemplo de la ley de Gustafson-Barsis, Formalización matemática. Comparación entre Amdahl y Gustafson-Barsis. Análisis de escalabilidad: Escalabilidad en sistemas paralelos, Ejemplo de análisis de escalabilidad: Análisis de los tiempos de ejecución, Análisis de Speedup (Amdahl y Gustafson-Barsis), Análisis de Eficiencia (escalabilidad). Modelo de isoeficiencia: Modelo de carga fija y modelo de tiempo fijo, Función de isoeficiencia.

9 Ejecución paramétrica

10 Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa. Speedup heterogéneo. Eficiencia heterogénea. Overhead heterogéneo.

Caso práctico en sistemas paralelos heterogéneos: Cálculo de la potencia de cómputo relativa, Cálculo del Speedup heterogéneo ideal, Distribución de carga, Limitaciones y consideraciones.



1

Métricas en los sistemas paralelos

Métricas en los sistemas paralelos

4

- Utilizamos métricas para evaluar el comportamiento de un sistema.
- Habitualmente, analizamos **algoritmos secuenciales de manera teórica** a partir del comportamiento asintótico.
 - Se estima $T(n)$ a medida que crece el tamaño de problema y es idéntico en cualquier plataforma:

$$O(n) \quad O(n^x) \quad O(\log n) \quad O(n \log n) \quad O(2^n)$$

- En la **práctica**, algoritmos distintos que resuelven el mismo problema y tienen el mismo comportamiento asintótico, pueden alcanzar tiempos de ejecución diferentes en la misma arquitectura debido a otros factores (Por ejemplo: Localidad).



Métricas en los sistemas paralelos

5

- El análisis **teórico** de un **algoritmo paralelo** requiere considerar:
 - Número de unidades de procesamiento
 - Costos de Sincronización y Comunicación
 - Balance de carga
 - Hardware y sistema operativo
- En Sistemas Paralelos, interesa saber cual es la ganancia obtenida respecto a la ejecución de la solución secuencial.
 - ¿Cuál es la ganancia que se obtiene?
¿Siempre se obtiene una ganancia?
 - ¿Qué indica que un sistema paralelo es mejor que otro?
 - ¿Qué ocurre al aumentar...?
Unidades de procesamiento
Tamaño del problema



Punto de vista empírico: basándonos en tiempo de ejecución y métricas derivadas. Inicialmente, suponemos un **sistema paralelo homogéneo**.

A dark, industrial setting, possibly a server room or a control room. In the foreground, a desk is cluttered with various items: an open book with dense text, a pen, and some papers. In the background, there are racks of electronic equipment with glowing orange lights, suggesting a complex system or a laboratory. The overall atmosphere is mysterious and technical.

2

Tiempo de ejecución

Métricas

Tiempo de ejecución

7

- **Tiempo de ejecución:**



- **Del algoritmo secuencial (T_s):** Tiempo transcurrido entre el comienzo y la finalización de la ejecución del programa secuencial
- **Del algoritmo paralelo (T_p):** Tiempo transcurrido entre el comienzo y la finalización del último hilo o proceso del programa paralelo

- **Dos formas de medir el tiempo de ejecución:**

- Llamada al sistema operativo que interactúa con el **reloj del sistema**.
- Métricas asociadas al hardware.

Variabilidad en el tiempo obtenido en cada ejecución:

- Ejecutar varias veces y obtener el tiempo **promedio**
Imposibilidad cuando una única ejecución demora horas
- Estimaciones (**Firma**)





3

Throughput

Throughput

9

Throughput (rendimiento efectivo): Cantidad de trabajo u operaciones que se completan por unidad de tiempo.

- Ejemplos:

- **FLOPS:** Operaciones de punto flotante por segundo.
- **IOPS:** Operaciones enteras por segundo.
- **TOPS:** Tera operaciones por segundo.
 - Aceleradores e IA: operaciones aritmético-lógicas enteras de baja precisión (Int8, Int4).
- **MIPS:** Millones de instrucciones por segundo.
- **Mbps o MBps:** Megabits o Megabytes por segundo, respectivamente.
- **OPS/W:** Operaciones por segundo por cada watt consumido.
- **FLOPS/W:** Operaciones de punto flotante por segundo por cada watt consumido.



Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup

11

Speedup (aceleración): Beneficio relativo obtenido al ejecutar un programa después de una mejora u optimización.

- Suponiendo un programa al cual se le hizo alguna mejora. El **Speedup** se calcula:



$$S = \frac{T_a}{T_b}$$

T_a : tiempo antes de la mejora

T_b : tiempo después de la mejora

Speedup paralelo: Beneficio relativo obtenido al ejecutar un algoritmo después de paralelizarlo.

- El **Speedup** paralelo utilizando P unidades de procesamiento se calcula:

$$S(P) = \frac{T_s}{T_p(P)}$$

T_s : Tiempo de ejecución del algoritmo secuencial.

$T_p(P)$: Tiempo de ejecución del algoritmo paralelo ejecutado con P unidades de procesamiento.

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup absoluto y relativo

14

Speedup absoluto: Cuando el tiempo de ejecución secuencial de referencia es el del mejor algoritmo secuencial conocido para ese problema.

$$S_{\text{abs}}(P) = \frac{T_{\text{mejor}}}{T_p(P)}$$

Speedup relativo: Cuando el tiempo de ejecución secuencial de referencia es el de la versión secuencial disponible, no necesariamente la mejor, o el tiempo de ejecución del programa paralelo ejecutado con un solo hilo o proceso.

$$S_{\text{rel}}(P) = \frac{T_{\text{ref}}}{T_p(P)}$$



$$S_{\text{rel}}(P) = \frac{T(1)}{T_p(P)}$$

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup degradado: Cuando el valor de Speedup es menor igual a 1 y no se obtiene ningún beneficio de la paralelización.

- Cuando el tiempo de ejecución paralela es mayor al tiempo de ejecución secuencial.
- Problemas “*chicos*” con tiempos de ejecución secuencial despreciable.
 - *Overhead* paralelo (creación de hilos, sincronización, comunicación...) impacta en el rendimiento.



No siempre se obtienen beneficios al paralelizar!!!



Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup

Speedup lineal (ideal o perfecto), sublineal y superlineal

- Tres casos cuando el Speedup es mayor a 1 ($S(P) > 1$):



- **Speedup lineal (ideal o perfecto):** $S(P) = P$
- **Speedup sublineal:** $S(P) < P$
- **Speedup superlineal:** $S(P) > P$

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup

Speedup lineal (ideal o perfecto)

20

Speedup lineal (ideal o perfecto): Cuando el valor de Speedup es igual al número de unidades de procesamiento utilizadas ($S(P)=P$).

- **Mejor caso:** distribuimos el trabajo equitativamente entre **P unidades de procesamiento** y el programa paralelo correrá P veces más rápido que el programa secuencial:

$$T_p(P) = \frac{T_s}{P}$$



$$S(P) = \frac{T_s}{T_p(P)} = \frac{T_s}{\frac{T_s}{P}} = P$$



Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

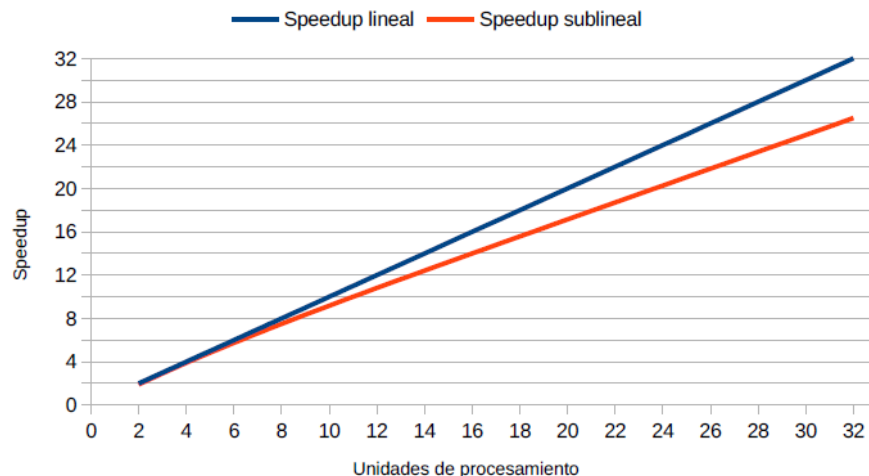
Speedup

Speedup sublineal

22

Speedup sublineal: Cuando se obtiene beneficio de la paralelización ($S(P) > 1$) pero el valor de Speedup es menor al número de unidades de procesamiento utilizadas ($S(P) < P$).

- Caso habitual, debido a características (*overhead*) inherentes al algoritmo paralelo:
 - *Startup*
 - Sincronización y comunicación
 - Esperas
 - Desbalance de carga
 - Cuellos de botella
 - Contención
 - Cómputo redundante



Características que no posee un algoritmo secuencial !!!



Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup

Speedup superlineal

24

Speedup superlineal: Cuando se obtiene un Speedup mayor al número de unidades de procesamiento utilizadas ($S(P) > P$).

- En la mayoría de los casos el Speedup es sublineal o cercano al ideal.



- **Speedup superlineal: Situación poco frecuente!!!**

- **Speedup superlineal puede darse:**



- Algoritmo secuencial no óptimo.
- Diferencias en la carga de trabajo.
- Efectos debidos a la arquitectura.

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

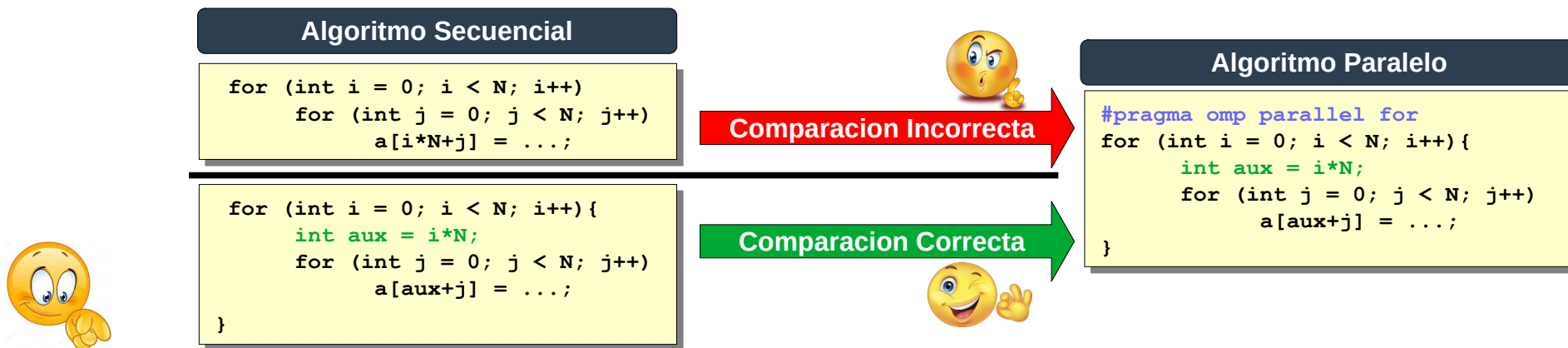
Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

Speedup

Speedup superlineal - Algoritmo secuencial no óptimo

- Dos situaciones:
 - **Diferentes estrategias:** No contamos con el código del algoritmo secuencial y paralelizamos desconociendo la estrategia.
 - **Optimización asimétrica:** Contamos con el código del algoritmo secuencial, paralelizamos, optimizamos la parte “secuencial” del algoritmo paralelo pero no en la versión secuencial.



En problemas “regulares” es extraño o imposible obtener speedups superlineales. Si lo obtenemos, el problema suele estar en el algoritmo secuencial y no en el algoritmo paralelo!!!

Prueba: Si el algoritmo paralelo lo permite, correr con un solo hilo o proceso y usarlo como referencia.

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

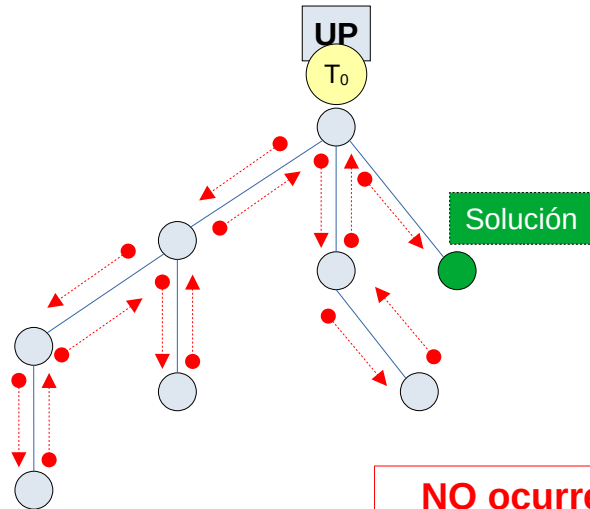
Efectos debidos a la arquitectura

Speedup

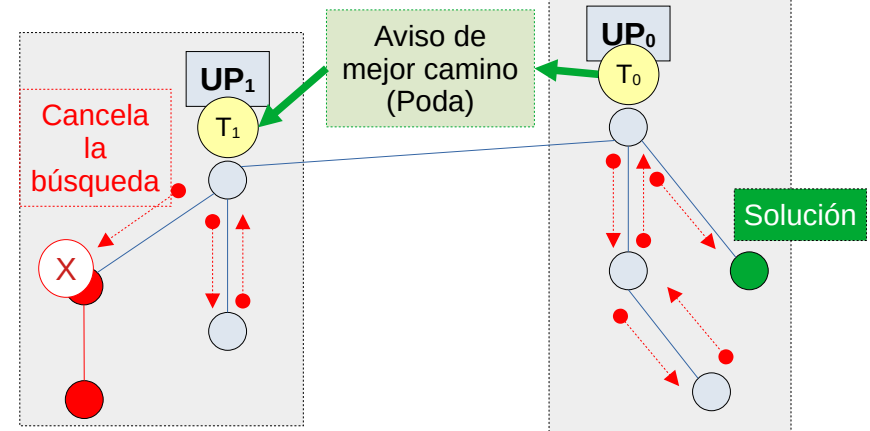
Speedup superlineal – Diferencias en la carga de trabajo

- La versión paralela del algoritmo realiza menos trabajo que la versión secuencial:
 - Búsquedas en grafos (*backtracking*, *branch and bound*)
 - Optimización combinatoria
 - Poda.

Algoritmo Secuencial



Algoritmo Paralelo



NO ocurre en algoritmos regulares!!!

Speedup

Speedup absoluto y relativo

Speedup degradado

Speedup lineal (ideal o perfecto), sublineal y superlineal

Speedup lineal (ideal o perfecto)

Speedup sublineal

Speedup superlineal:

Algoritmo secuencial no óptimo

Diferencias en la carga de trabajo

Efectos debidos a la arquitectura

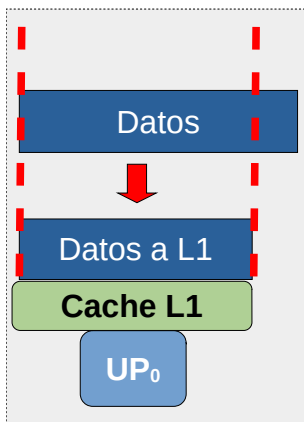
Speedup

Speedup superlineal - Efectos debidos a la arquitectura

- Uso diferente de la memoria cache:

Algoritmo Secuencial

El volumen de datos a procesar podría no caber en memoria cache.

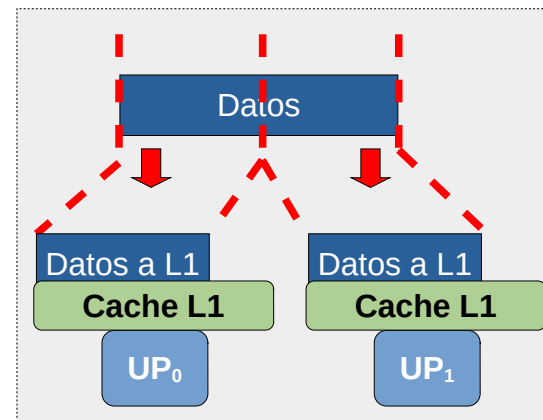


Mas fallos de cache!!!



Algoritmo Paralelo

El volumen de datos a procesar se divide en porciones entre las unidades de procesamiento y cada porción podría almacenarse por completo en las caches de cada unidad de procesamiento.



Menos fallos de cache!!!



5

Eficiencia

Eficiencia: Porcentaje del tiempo en que las unidades de procesamiento realizan trabajo útil en un algoritmo paralelo.

$$E(P) = \frac{S(P)}{P}$$

$S(P)$: Speedup utilizando P unidades de procesamiento.
 P : Unidades de procesamiento utilizadas.

$0 < E(P) < 1$
Cuando $S(P) < P$

- **También se define:** Relación entre el *speedup* obtenido $S(P)$ y el *speedup* ideal $S_{ideal}(P)$:

$$E(P) = \frac{S(P)}{S_{ideal}(P)}$$



6

Balance de carga

Balance de carga

34

Balance de carga: Mide qué tan equitativamente se distribuye el trabajo entre los hilos o procesos en un algoritmo paralelo.

- Cuando algunos hilos o procesos realizan más (o menos) trabajo que otros.



$$B = \frac{\textit{Promedio}(T)}{\textit{Máximo}(T)}$$

***Promedio*(T):** Promedio de los tiempos de ejecución de los hilos o procesos.

***Máximo*(T):** Tiempo de ejecución del hilo o proceso que más tardó.

- La carga estará más balanceada cuando la métrica tienda a 1
- La carga estará más desbalanceada cuando la métrica tienda a 0

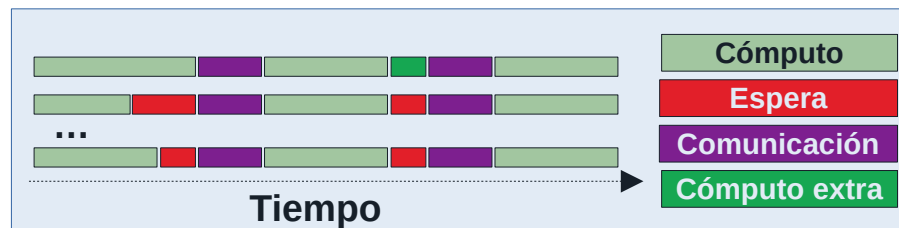


Overhead

36

Overhead: Trabajo adicional, medido como tiempo total de cómputo acumulado por todas las unidades de procesamiento, que realiza un algoritmo paralelo debido al paralelismo, que no se lleva a cabo en el algoritmo secuencial.

$$T_O = P \cdot T_p(P) - T_s$$



P : Unidades de procesamiento utilizadas.

$T_p(P)$: Tiempo de ejecución paralelo ejecutado con P unidades de procesamiento.

T_s : Tiempo de ejecución secuencial.

- El óptimo u **overhead mínimo** es 0 y ocurre cuando el speedup es ideal:

$$T_O = P \cdot T_p(P) - T_s = P \cdot \frac{T_s}{P} - T_s = 0$$

Overhead

37

- Un algoritmo paralelo termina antes que el secuencial, pero realiza más trabajo.
- Ejemplo (asumimos $P=4$):

$$T_s = 100$$

$$T_p(4) = 30$$

El algoritmo paralelo termina antes.

Calculamos el trabajo:

$$W_s = T_s = 100$$

$$W_p = P \cdot T_p(4) = 4 \cdot 30 = 120$$



$$T_o = W_p - W_s = 20$$



Trabajo adicional en el algoritmo **paralelo** ausente en el secuencial:

Creación de hilos o procesos
Comunicación / Sincronización
Esperas
...

Reducir el tiempo de ejecución no implica menos trabajo.

Overhead

38

Overhead absoluto

$$T_O = P \cdot T_p(P) - T_s$$

Overhead relativo

$$O_{rel} = \frac{T_O}{W}$$



$$W = W_s = T_s$$

Carga de trabajo - Tamaño del problema

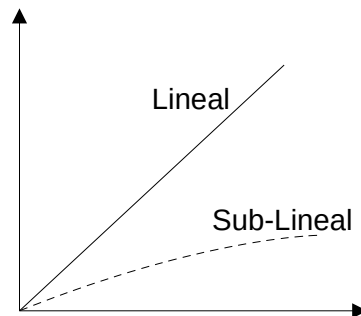
- **Overhead absoluto crece:**

- **Con una componente que depende linealmente de (P):**

- Hilos o procesos introducen una parte lineal no paralelizable.
 - Inactividad de unidades de procesamiento.
 - Comunicación entre unidades de procesamiento.

- **Sub-linealmente en función de la carga de trabajo (W)**

- Crece mas lento que W , **el overhead relativo disminuye.**



T_O representa una fracción cada vez menor a medida que aumenta W .



- Overhead absoluto en función de P y W : $T_O(W, P) = P \cdot T_p(P) - W$

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad: Propiedad deseable de un sistema, red o proceso, que indica su habilidad para reaccionar y adaptarse sin perder calidad, manejar el crecimiento continuo de trabajo y estar preparado para escalar sin perder calidad en los servicios ofrecidos.

- **En Sistemas Paralelos:** no alcanza con que el algoritmo funcione. Se estudia la capacidad del sistema o algoritmo para utilizar más recursos sin degradar su rendimiento.
 - Ley de Amdahl
 - Ley de Gustafson
 - Analisis de escalabilidad
 - Modelo de Isoeficiencia



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

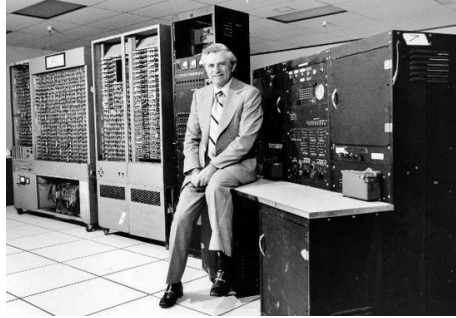
Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ley de Amdahl

42



- En 1967 Gene Amdahl hizo una observación que se conoce como la Ley de Amdahl:

Ley de Amdahl: Para un programa con una fracción secuencial fija, el *Speedup* máximo alcanzable está limitado por esta fracción. En consecuencia, a menos que prácticamente todo el programa sea paralelizable, el aumento de rendimiento será muy limitado, independientemente del número de procesadores* disponibles.

* En términos modernos, entiéndase procesadores como unidades de procesamiento.

G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In AFIPS Conference Proceedings, vol. 30, 483–485, AFIPS Press, Reston, VA, 1967.

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

- Suponemos:
 - Algoritmo secuencial: tiempo de ejecución para una carga de trabajo W , $T_s=20$
 - **90% paralelizable**: fracción paralela $f_p=90\%$.
 - **10% NO paralelizable**: fracción secuencial no paralelizable $f_s=10\%$.

Tiempo de ejecución de las fracciones

$$T_{fs} = 0,1 \cdot T_s = 0,1 \cdot 20 = 2$$

$$T_{fp}(P) = \frac{0,9 \cdot T_s}{P} = \frac{0,9 \cdot 20}{P} = \frac{18}{P}$$



Tiempo de ejecución paralelo

$$T_p(P) = T_{fs} + T_{fp}(P) = 2 + \frac{18}{P}$$

Límite del Speedup **ideal** a medida que P crece

$$\lim_{P \rightarrow \infty} S(P) = \lim_{P \rightarrow \infty} \frac{20}{2 + \frac{18}{P}} = 10$$



Paralelizando perfectamente el 90%, no importa el valor de P, nunca el Speedup es mejor que 10!!!

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis,

Formalización matemática.

Comparación entre Amdahl y Gustafson-Barsis.

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

- Si tenemos un problema con una **carga de trabajo fija** (W), la **fracción secuencial** (f_s) **limita el *Speedup* máximo** que podemos alcanzar:
 - Aunque agreguemos más unidades de procesamiento, nunca superaremos el inverso de la fracción secuencial:



$$\lim_{P \rightarrow \infty} S_{Max}(P) = \lim_{P \rightarrow \infty} \frac{1}{f_s + \frac{(1-f_s)}{P}} = \frac{1}{f_s}$$



Alternativa usando la fracción paralelizable

$$\lim_{P \rightarrow \infty} S_{Max}(P) = \lim_{P \rightarrow \infty} \frac{1}{(1-f_p) + \frac{f_p}{P}} = \frac{1}{(1-f_p)}$$



¿Es muy malo un Speedup tan pequeño?

En muchos casos, obtener una aceleración de 5 o 10 es aceptable.



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis.

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ley de Gustafson

48

- **En la práctica:** *Speedups* muy altos con miles de unidades de procesamiento!!!



En 1988, Gustafson y Barsis observan qué ocurre cuando aumentan el tamaño del problema y a la vez el número de unidades de procesamiento.

Ley de Gustafson–Barsis: Si el tamaño del problema aumenta a medida que se incrementa la cantidad de unidades de procesamiento, el impacto de la fracción secuencial del programa se vuelve cada vez menos importante. En este escenario, el *Speedup* puede crecer casi linealmente con el número de unidades de procesamiento, aun cuando exista una fracción del código que no puede paralelizarse.

J.L. Gustafson, Reevaluating Amdahl's law, Commun. ACM 31 (5) (1988) 532–533.



**La ley de Amdahl no tiene en cuenta el tamaño del problema!!!
En 1967, se pensaba en resolver problemas fijos**



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

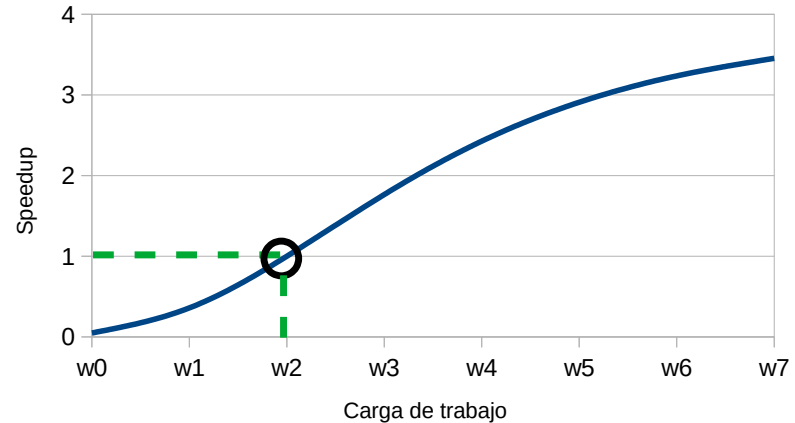
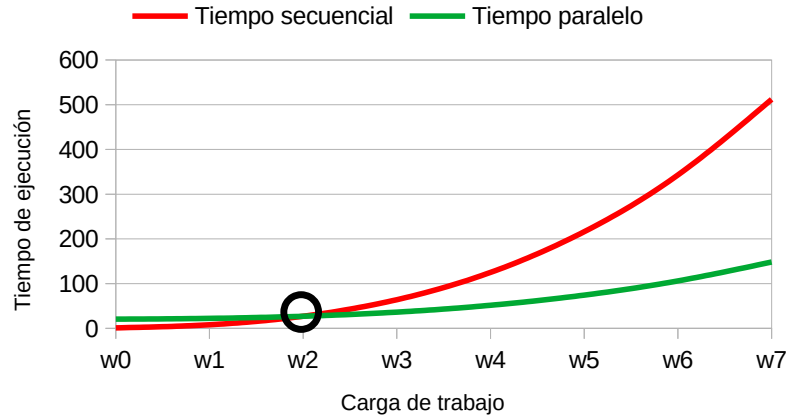
Función de isoeficiencia

Escalabilidad

Ley de Gustafson - Ejemplo

50

- **Suponemos:** un problema genérico pero representativo, P fijo ($P=4$) y cargas de trabajo crecientes ($w_0, w_1 \dots w_n$)
 - **Cargas de trabajo muy chicas:** tiempos despreciables, *overhead* alto, la fracción secuencial domina. El tiempo paralelo no obtiene mejoras ($Speedup < 1$)
 - **Cargas de trabajo mas grandes:** *overhead* casi no cambia, la fracción paralela crece y el *Speedup* aumenta.



Sólo tiene sentido paralelizar el algoritmo a partir de la carga de trabajo w_3 ($speedup > 1$)
Se acerca al ideal pero la Ley no predice que lleguemos a este.

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

- Ahora, consideramos la carga de trabajo (W) como una variable del *Speedup*.

- Por lo tanto, el límite: $\lim_{W \rightarrow \infty} S_p(W, P) = P$



- Fórmula de **Speedup escalado** de Gustafson-Barsis:

$$S_{GyB}(P) = \underbrace{f_s}_{\text{Carga fija}} + \underbrace{(1-f_s)}_{\text{Carga creciente}} \cdot P = T_{fs} + P \cdot T_{fp}(P)$$

Parten de un sistema paralelo y se preguntan ¿Cuánto tardaría una única unidad de procesamiento en hacer todo el trabajo del algoritmo paralelo?

Si el problema es lo suficientemente grande, la fracción secuencial (f_s) se convierte en una fracción minúscula de la carga total de trabajo, mientras que la fracción paralela ($1-f_s$) es mayor, permitiendo que el *Speedup* crezca hacia P .



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

- Suponemos un problema de **tamaño fijo** y una fracción paralela del 10% ($f_s=0,1$).

- Según Amdahl:

$$S(P) = \frac{1}{f_s} = \frac{1}{0,1} = 10$$

¿Cómo disminuye f_s ?

No lo dice...

No considera la carga de trabajo



Unidades de procesamiento	Speedup S(P) según Amdahl
1	1
2	$S(2) = \frac{1}{0,1 + \frac{0,9}{2}} \approx 1,82$
4	$S(4) = \frac{1}{0,1 + \frac{0,9}{4}} \approx 3,08$
8	$S(8) = \frac{1}{0,1 + \frac{0,9}{8}} \approx 4,71$
16	$S(16) = \frac{1}{0,1 + \frac{0,9}{16}} \approx 6,40$

$$S_{(P)} = \frac{1}{f_s + \frac{(1-f_s)}{P}}$$

- Según Gustafson-Barsis, sólo si la carga de trabajo aumenta $f_p = 1 - f_s$ crece mas rápido y f_s disminuye (por ejemplo 0,02), :

$$S(16) = f_s + (1 - f_s) \cdot 16 = 0,02 + (1 - 0,02) \cdot 16 = 15,70$$



Leyes de **Amdahl** y de **Gustafson-Barsis** son **complementarias**



Amdahl explica los **límites** para una **carga de trabajo fija**

“Si el problema es fijo existe un límite en el cual no puedo agregar mas unidades de procesamiento”

Gustafson-Barsis explica cómo puede alcanzarse **mayor beneficio**

“Si el problema crece podemos agregarse mas unidades de procesamiento y obtener beneficios”



... aunque existen excepciones

Existen problemas donde no podemos incrementar el tamaño de problema (rige la ley de Amdahl)



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Análisis de escalabilidad

57

- **Objetivo del análisis de escalabilidad:** estudiar cómo se comporta un algoritmo o sistema paralelo al incrementar su capacidad de cómputo, aumentando la cantidad de unidades de procesamiento o el tamaño de la carga de trabajo.
- A partir de la ejecución del algoritmo paralelo:
 - **Tiempos de ejecución:** Viabilidad del algoritmo paralelo.
 - **Speedup:** Speedup ideal, Ley de Amdahl y Gustafson-Barsis.
 - **Eficiencia:** Escalabilidad fuerte y débil. Qué tan bien usamos los recursos.



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Análisis de escalabilidad - Escalabilidad en sistemas paralelos

59

Sistema paralelo escalable: Un sistema paralelo es escalable si la combinación de la arquitectura y los algoritmos que ejecuta permite mantener la eficiencia aproximadamente constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema.

- Definición en el sentido más amplio para analizar el sistema paralelo completo.
 - Conclusiones al variar arquitectura, red e interconexiones, topologías, memoria, latencias, o sistemas paralelos diferentes.
- Solo interesa analizar cómo se comporta un algoritmo paralelo:

Programa paralelo escalable: Un programa paralelo es escalable si la eficiencia se mantiene aproximadamente constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema.



- **Tipos de escalabilidad de un algoritmo paralelo:**
 - **Escalabilidad débil (*ídem definición de programa paralelo escalable*):** Un programa paralelo es débilmente escalable (o simplemente escalable) si la eficiencia se mantiene aproximadamente constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema.
 - **Escalabilidad fuerte:** Un programa paralelo es fuertemente escalable si la eficiencia se mantiene aproximadamente constante al incrementar el número de unidades de procesamiento sin aumentar el tamaño del problema.
 - **Escalabilidad limitada o nula:** Un programa paralelo se considera no escalable cuando la eficiencia disminuye significativamente al aumentar la cantidad de unidades de procesamiento, independientemente de si se incrementa o no el tamaño del problema.



La **escalabilidad** define la **capacidad** de un programa **de no degradar el rendimiento**.
Si la **eficiencia no es constante pero aumenta** es **escalable** (*superescalable*)!!!

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ejemplo de análisis de escalabilidad

62

- Analizamos la **escalabilidad** del **programa paralelo**.
- Contamos con el mejor algoritmo secuencial.
 - La ejecución del algoritmo secuencial sobre una única unidad de procesamiento ($UP=1$)
- Carga de trabajo inicial $W=64$ e incrementamos en potencias de 2 hasta $W=8192$.
- Ejecutamos el algoritmo paralelo con $UP=2$ unidades de procesamiento e incrementamos en potencias de 2 hasta $UP=16$.
- Tres tablas (todas las combinaciones $W \times UP$):
 - Tiempos de ejecución.
 - Speedups.
 - Eficiencias.

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ejemplo de análisis de escalabilidad – Tiempos de ejecución

64

Tiempo (seg)	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	0,38	0,90	2,05	4,61	10,24	22,53	49,15	106,50
2	0,43	0,75	1,58	3,39	6,56	12,52	26,71	57,26
4	0,40	0,62	1,14	2,09	3,66	7,04	13,81	29,26
8	0,48	0,62	0,88	1,37	2,42	3,56	6,75	14,79
16	0,48	0,62	0,85	1,15	1,56	1,83	3,45	7,40

$W=64$ carga chica

Sin beneficios

$$T_s < T_p(P)$$

Overhead > Cómputo



Sólo con el tiempo de ejecución ¿Podemos responder?



¿Qué tan significativa es la mejora?

¿Es posible optimizar aún más el algoritmo?

¿Hasta que valores se pueden reducir los tiempos de ejecución?

¿Qué tan eficientemente se están aprovechando los recursos?

NO!!! Necesitamos otras métricas (*Speedup* y Eficiencia)

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ejemplo de análisis de escalabilidad – Speedup

66

Speedup	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,90	1,20	1,30	1,36	1,56	1,80	1,84	1,86
4	0,96	1,44	1,80	2,20	2,80	3,20	3,56	3,64
8	0,80	1,44	2,32	3,36	4,24	6,32	7,28	7,20
16	0,80	1,44	2,40	4,00	6,56	12,32	14,24	14,40

$W=64$

Speedup degradado

$$S(P) < 1$$



$W=128, 256, 512$ y 1024

Tiempo de ejecución muestra mejoras

Speedup dice que no son significativas

Amdahl

Carga de trabajo fija

Speedup deja de crecer

Se aleja del ideal

Usar sólo UPs necesarias

Cálculo de f_s



Gustafson-Barsis

Carga de trabajo aumenta

Speedup se acerca al ideal

$$S(P) = \frac{1}{f_s + \frac{(1-f_s)}{P}}$$

$$f_s = \frac{\frac{P}{S(P)} - 1}{P - 1}$$

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Ejemplo de análisis de escalabilidad – Eficiencia

68

Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90

$W=64$

Uso de recursos

$E(P) < 45\%$

No tiene sentido paralelizar



Mas adecuado analizar la escalabilidad en bloques

Escalabilidad

Ejemplo de análisis de escalabilidad – Eficiencia

69

Speedup	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,90	1,20	1,30	1,36	1,56	1,80	1,84	1,86
4	0,96	1,44	1,80	2,20	2,80	3,20	3,56	3,64
8	0,80	1,44	2,32	3,36	4,24	6,32	7,28	7,20
16	0,80	1,44	2,40	4,00	6,56	12,32	14,24	14,40

Error común

Tomar decisiones mirando Speedup



Speedup se duplica (parece bueno!!!)



$W=128, 256, 512 \text{ y } 1024$

No es fuertemente escalable
Tampoco débilmente escalable



Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90

La eficiencia decae, nos dice que no escala
No se usan bien los recursos!!!



Escalabilidad

Ejemplo de análisis de escalabilidad – Eficiencia

70

Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90

$W=512, 1024$ y 2048

No es fuertemente escalable



$w=4096$

Fuertemente escalable en torno a 90%

Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90

Eficiencia aumenta, no se degrada
Débilmente escalable



Nota:

Débilmente escalable aún con eficiencia baja.



Eficiencia	W	
UP	512	1024
1	1,00	1,00
8	0,42	0,53
16	0,25	0,41

Escalabilidad

Ejemplo de análisis de escalabilidad – Eficiencia

71

Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90



$w=8192$

Fuertemente escalable en torno a 90%

Eficiencia	Carga de trabajo (W)							
UP	64	128	256	512	1024	2048	4096	8192
1	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
2	0,45	0,60	0,65	0,68	0,78	0,90	0,92	0,93
4	0,24	0,36	0,45	0,55	0,70	0,80	0,89	0,91
8	0,10	0,18	0,29	0,42	0,53	0,79	0,91	0,90
16	0,05	0,09	0,15	0,25	0,41	0,77	0,89	0,90



Eficiencia aumenta

Luego, se mantiene en torno a 90%

Débilmente escalable

¿Si crece $W=16384$ y $UP=32$?

Podemos asumir que seguirá escalando

Si la eficiencia decae, no lo haría de forma abrupta

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Modelo de isoeficiencia

73

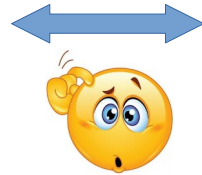
¿Cuánto debe crecer W para mantener la eficiencia constante al agregar UPs?



Isoeficiencia → Modelo de isoeficiencia → Función de isoeficiencia

¿Están relacionadas?

Ley de Amdahl
Ley de Gustafson-Barsis



Escalabilidad débil
Escalabilidad fuerte

Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

Escalabilidad

Modelo de isoeficiencia - Modelo de carga fija y modelo de tiempo fijo

- Las leyes de **Amdahl** y **Gustafson-Barsis** se conocen como **modelos de Speedup**.
 - **Modelo de carga fija (Amdahl)** - Asociado con **escalabilidad fuerte**:
 - La carga de trabajo es fija.
 - La fracción secuencial no disminuye, domina y limita el *Speedup*.
 - La eficiencia decae con más unidades de procesamiento. No es fuertemente escalable.
 - **Modelo de tiempo fijo (Gustafson-Barsis)** - Asociado con **escalabilidad débil**:
 - Aplicaciones que requieren mayor precisión.
 - Importa resolver problemas más grande con más unidades de procesamiento sin que aumente el tiempo de ejecución paralelo (tiempo de ejecución aproximadamente fijo)

P unidades de procesamiento ¿Cuándo se alcanza un *Speedup* ideal?

- **Modelo de carga fija:** cuando el problema se resuelve P veces más rápido que secuencial.
- **Modelo de tiempo fijo:** cuando puede resolverse un problema P veces más grande en la misma cantidad de tiempo que secuencial.



Modelo de isoeficiencia formaliza lo anterior a partir de la **Función de isoeficiencia**

Escalabilidad

Modelo de isoeficiencia - Modelo de carga fija y modelo de tiempo fijo

76

Modelo de tiempo fijo?
Gustafson?
Escalabilidad débil?



Supongamos:

Problema secuencial donde el costo es UNA unidad de tiempo por UNIDAD de la carga de trabajo.

Si $W=n$ entonces $T_s(W) = n$

Débilmente escalable con una eficiencia del 90% (0,90).



Eficiencia	W		
UP	512	1024	2048
1	1,00	1,00	1,00
8	0,90		
16		0,90	
32			0,90

$$E = \frac{S}{P}$$

$$S = E \cdot P$$

Speedup	W		
UP	512	1024	2048
1	1,00	1,00	1,00
8	7,20		
16		14,40	
32			28,80

$$S = \frac{T_s}{T_p}$$

$$T_p = \frac{T_s}{S}$$

Tiempo (seg)	W		
UP	512	1024	2048
1	512,00	1024,00	2048,00
8	71,11		
16		71,11	
32			71,11



Escalabilidad

Ley de Amdahl:

Ejemplo de la ley de Amdahl

Formalización matemática

Ley de Gustafson-Barsis:

Ejemplo de la ley de Gustafson-Barsis

Formalización matemática

Comparación entre Amdahl y Gustafson-Barsis

Análisis de escalabilidad:

Escalabilidad en sistemas paralelos

Ejemplo de análisis de escalabilidad:

Análisis de los tiempos de ejecución

Análisis de Speedup (Amdahl y Gustafson-Barsis)

Análisis de Eficiencia (escalabilidad)

Modelo de isoeficiencia:

Modelo de carga fija y modelo de tiempo fijo

Función de isoeficiencia

$$W = \frac{E(W, P)}{1 - E(W, P)} \cdot T_o(W, P) = K \cdot T_o(W, P)$$

$$W = K \cdot T_o(W, p)$$



Función de isoeficiencia: muestra cómo debe crecer la carga de trabajo W al aumentar P para mantener una eficiencia $E(W, P)$ constante, dado un *overhead* $T_o(W, P)$.

Cuanto menor sea mejor!!!

- El **overhead absoluto** tiende a crecer con una componente **lineal** que depende de P .
- En algoritmos escalables, el *overhead* absoluto crece de manera **sub-lineal** (más lento) respecto a W , el **overhead relativo disminuye** a medida que el tamaño del problema aumenta.

El overhead crece mas rápido al incrementar P que al incrementar W

- Supongamos dos problemas:

Problema 1

- El overhead crece linealmente con un orden $2p \cdot \log_2(p) \rightarrow T_o(W,p) = 2p \cdot \log_2(p)$
- Función de isoeficiencia (**chica**):

$$W = K \cdot T_o(W,p) = K \cdot 2p \cdot \log_2(p)$$

- Para $p=16 \rightarrow W=128.K$
- Usar 32 unidades de procesamiento ($p=32$) requiere que $W=320.K$ para mantener la eficiencia constante.
- W tiene que crecer en un factor de 2,5:

$$320.K = 2,5 \cdot 128.K$$

W tiene que crecer un poco mas del doble



Problema 2

- El overhead crece **NO linealmente** con un orden $2^p \rightarrow T_o(W,p) = 2^p$
- Función de isoeficiencia (**grande**):

$$W = K \cdot T_o(W,p) = K \cdot 2^p$$

- Para $p=16 \rightarrow W=2^{16}.K$
- Usar 32 unidades de procesamiento ($p=32$) requiere que $W=2^{32}.K$ para mantener la eficiencia constante.
- W tiene que crecer en un factor de 2^{16} :

$$2^{32}.K = 2^{16} \cdot 2^{16}.K$$

W tiene que aumentar 65536 veces



En el **problema 2** asumimos que el overhead tiene orden 2^p , sólo para ejemplificar. Recordar, el overhead crece linealmente en función de p y sub-linealmente en función de w .

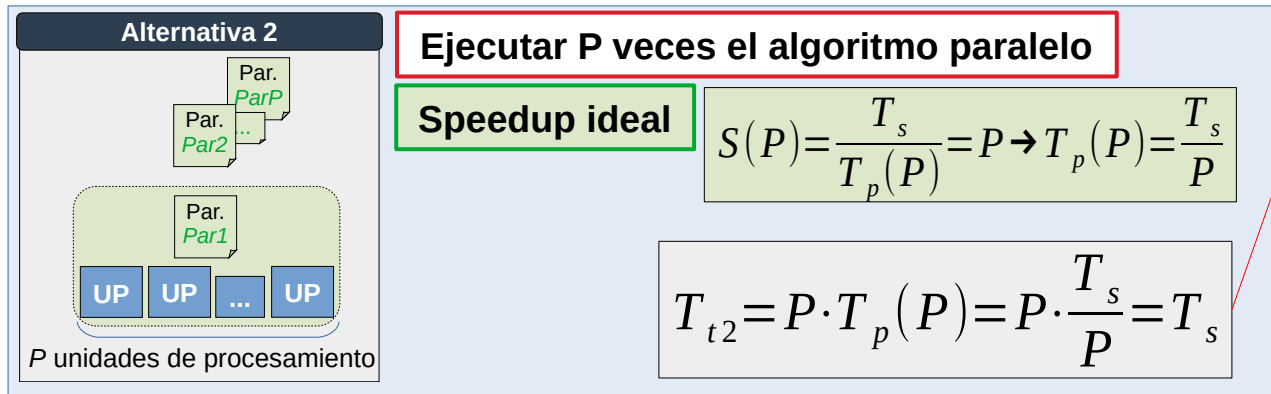
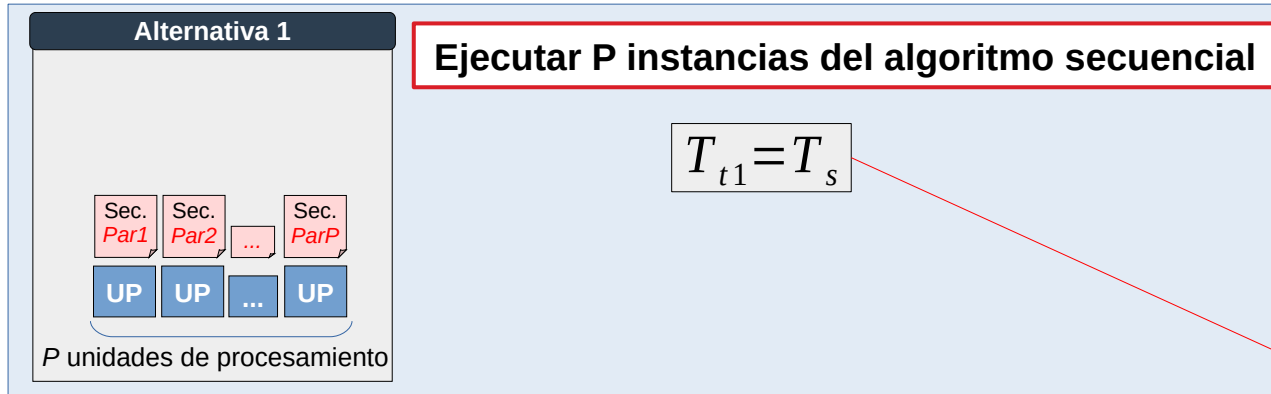




Ejecución paramétrica

81

- Aplicación que se ejecuta varias veces sobre una arquitectura variando parámetros.
- En una arquitectura paralela con P unidades de procesamiento, existen dos alternativas:



Mismo tiempo de ejecución total
¿Conviene paralelizar?

Casos convenientes:

- Speedup superlineal
- Datos parciales
- Tiempo real
- Simulaciones
- ...



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

83

- **Sistema heterogéneo:** arquitecturas paralelas compuestas por unidades de procesamiento con características diferentes.
 - Clusters con nodos diferentes
 - Multicores asimétricos
 - Híbridos: Clusters de multicores con GPUs
 - ...



Problema: ¿Cómo distribuimos la carga de trabajo?

**¿Métricas?
¿Speedup ideal?**



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

85

- P unidades de procesamiento cada una con **distinta potencia de cómputo**:

$$pc_0, pc_1, \dots, pc_{P-1}$$

- pc_i : capacidad de cómputo de la unidad de procesamiento i
 - Cantidad de trabajo por unidad de tiempo
 - pc_{base} : la unidad de procesamiento mas potente
- **Potencia de cómputo relativa (pcr_i)**: potencia de cómputo de la unidad de procesamiento i respecto a la potencia de cómputo de la unidad mas potente

$$pcr_i = \frac{pc_i}{pc_{base}}$$



¿Cómo estimamos pc_i y pcr_i ?
Profiling para obtener el tiempo de ejecución

$$pc_i \propto \frac{1}{T_{S(i)}}$$



$$pcr_i = \frac{T_{S(base)}}{T_{S(i)}}$$

Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Speedup heterogéneo

87

- **Speedup heterogéneo:** beneficio del algoritmo paralelo respecto a la mejor ejecución secuencial (sobre la unidad base).

$$S_h(P) = \frac{T_{S(base)}}{T_p(P)}$$

- **Speedup ideal:** potencia de cómputo de la unidad de procesamiento i respecto a la potencia de cómputo de la unidad mas potente

$$S_{h(ideal)}(P) = \frac{\sum_{i=0}^{P-1} pc_i}{pc_{base}}$$

$$S_{h(ideal)}(P) = \sum_{i=0}^{P-1} pcr_i$$

Si el sistema es homogéneo las potencias son todas iguales
El Speedup ideal heterogéneo coincide con el Speedup ideal homogéneo



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Eficiencia Heterogénea

89

- **Eficiencia:** relación entre el Speedup obtenido y el Speedup ideal.
 - Heterogéneos:

$$E_h(P) = \frac{S_h(P)}{S_{h(ideal)}(P)}$$

- En función de la potencia de cómputo y potencia de cómputo relativa:

$$E_h(P) = \frac{pc_{base}}{\overline{pc}} \cdot \frac{T_{s(base)}}{P \cdot T_p(P)}$$

$$E_h(P) = \frac{1}{\overline{pcr}} \cdot \frac{T_{s(base)}}{P \cdot T_p(P)}$$

Si el sistema es homogéneo las potencias son todas iguales
La eficiencia heterogénea coincide con la eficiencia homogénea



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Overhead heterogéneo

91

- El trabajo paralelo debe ponderarse por la potencia de cómputo relativa:

$$\left(\sum_{i=0}^{P-1} pcr_i \right) \cdot T_p(P)$$

- Overhead heterogéneo (absoluto):** mide la diferencia entre el tiempo ponderado disponible y el tiempo secuencial de la unidad “base”.

$$T_{oh}(P) = \left(\sum_{i=0}^{P-1} pcr_i \right) \cdot T_p(P) - T_{s(base)}$$

Si el sistema es homogéneo todas las potencias relativas son iguales a 1
El *overhead* heterogéneo coincide con el *overhead* homogéneo



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Caso práctico en sistemas paralelos heterogéneos

93

- Antes de ejecutar la aplicación paralela en un sistema heterogéneo debemos:
 - 1) Calcular la potencia de cómputo relativa de cada unidad de procesamiento.
 - 2) Determinar el Speedup heterogéneo ideal.
 - 3) Distribuir la carga de trabajo entre las unidades de procesamiento según su potencia.
- **Profiling:** Ejecutamos una aplicación secuencial, con cierta carga de trabajo, sobre cada una de las 5 unidades de procesamiento de un sistema heterogéneo:

UP	Tiempo (seg)
P ₀	40
P ₁	24
P ₂	40
P ₃	30
P ₄	24



**P₁ y P₄ las unidades mas rápidas
Las mas potentes**

Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Caso práctico - Cálculo de la potencia de cómputo relativa

$$pcr_{P_0} = \frac{T_{s(base)}}{T_{s(P_0)}} = \frac{24}{40} = 0,6$$

$$pcr_{P_1} = \frac{T_{s(base)}}{T_{s(P_1)}} = \frac{24}{24} = 1$$

$$pcr_{P_2} = \frac{T_{s(base)}}{T_{s(P_2)}} = \frac{24}{40} = 0,6$$

$$pcr_{P_3} = \frac{T_{s(base)}}{T_{s(P_3)}} = \frac{24}{30} = 0,8$$

$$pcr_{P_4} = \frac{T_{s(base)}}{T_{s(P_4)}} = \frac{24}{24} = 1$$

UP	Tiempo (seg)	pcr
P ₀	40	0,6
P ₁	24	1
P ₂	40	0,6
P ₃	30	0,8
P ₄	24	1



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Caso práctico - Cálculo del Speedup heterogéneo ideal

97



Si fuera un Sistema Homogéneo $S_{ideal}(P) = 5$

En un Sistema heterogéneo...

$$S_{h(ideal)}(P) = \sum_{i=0}^{P-1} pcr_i$$

$$S_{h(ideal)}(P) = 0,6 + 1 + 0,6 + 0,8 + 1 = 4$$

UP	Tiempo (seg)	pcr
P ₀	40	0,6
P ₁	24	1
P ₂	40	0,6
P ₃	30	0,8
P ₄	24	1
		$S_{h(ideal)}(P) = 4$



Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Caso práctico - Distribución de carga

99

- Podemos **distribuir la carga de trabajo** entre las unidades de procesamiento de manera **estática**, según su **potencia de cómputo** relativa.
- Speedup** heterogéneo ideal equivale a la **capacidad total del sistema** (100%).
 - Potencia de cómputo relativa fracción de la capacidad total del sistema:

$$f_i = \frac{pcr_i}{S_{h(ideal)}(P)}$$

$$f_{P_0} = \frac{pcr_{P_0}}{S_{h(ideal)}(P)} = \frac{0,6}{4} = 0,15 = 15\%$$

$$f_{P_1} = \frac{pcr_{P_1}}{S_{h(ideal)}(P)} = \frac{1}{4} = 0,25 = 25\%$$

$$f_{P_2} = \frac{pcr_{P_2}}{S_{h(ideal)}(P)} = \frac{0,6}{4} = 0,15 = 15\%$$

$$f_{P_3} = \frac{pcr_{P_3}}{S_{h(ideal)}(P)} = \frac{0,8}{4} = 0,2 = 20\%$$

$$f_{P_4} = \frac{pcr_{P_4}}{S_{h(ideal)}(P)} = \frac{1}{4} = 0,25 = 25\%$$

UP	Tiempo (seg)	pcr	f_i - Porcentaje	Datos por UP
P ₀	40	0,6	0,15 – 15%	15
P ₁	24	1	0,25 – 25%	25
P ₂	40	0,6	0,15 – 15%	15
P ₃	30	0,8	0,20 – 20%	20
P ₄	24	1	0,25 – 25%	25
		$S_{h(ideal)}(P) = 4$	Total: 100%	Total: 100



Distribución de carga por cada 100 datos

Métricas en sistemas heterogéneos

Potencia de cómputo y potencia de cómputo relativa

Speedup heterogéneo

Eficiencia heterogénea

Overhead heterogéneo

Caso práctico en sistemas paralelos heterogéneos:

Cálculo de la potencia de cómputo relativa

Cálculo del Speedup heterogéneo ideal

Distribución de carga

Limitaciones y consideraciones

Métricas en sistemas heterogéneos

Caso práctico - Limitaciones y consideraciones

101

- Tiempo necesario para **profiling**.
- La potencia de cómputo depende del tipo de aplicación y carga de trabajo:
 - Unidad de procesamiento mas potente para una aplicación pero no para otras.
 - Unidad de procesamiento mas potente para cargas chicas pero no para cargas grandes.
 - Distorsiona la distribución estática (La carga paralela siempre es menor a la carga *profiling*).
- La estrategia de distribución estática es sólo una aproximación.
 - El problema de distribución balanceada es **NP-Completo**. 
 - **Preferible mapeo dinámico!!!** 

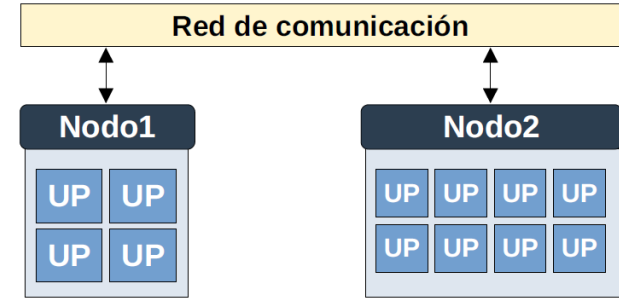
Métricas en sistemas heterogéneos

Caso práctico - Limitaciones y consideraciones

102



- Un cluster de nodos diferentes, cada nodo homogéneo.



- ¿Cómo calcular la potencia de cómputo? Opciones:
 - **De cada core individual:** En un nodo calcular la potencia de una unidad de procesamiento y multiplicarlo por el número de cores.
 - **NO: estamos asumiendo *Speedup* ideal, poco probable alcanzarlo.**
 - **Del nodo:** Ejecutar el algoritmo paralelo en el nodo para calcular la potencia.
 - **Coeficiente de eficiencia:** Ejecutar secuencial en un core y paralelo en el nodo. Calcular:

$$\eta = \frac{S(P)}{P}$$



$$pc_{nodo} = P \cdot pc_{core} \cdot \eta$$



Separa potencia teórica máxima de pérdidas reales por *overhead*

Próxima clase

Capítulo 10

Paradigmas de programación paralela



